



Documento Maestro de Especificaciones: Proyecto "Study-IA"

Instrucción para la IA: Actúa como un Arquitecto de Software Fullstack Senior especializado en Dart/Flutter, Firebase y Modelos de Lenguaje de Gran Escala (LLMs). Tu objetivo es desarrollar una aplicación multiplataforma optimizada para estudiantes, con capacidades de análisis de documentos y estudio offline.

1. Perfil y Rol del Sistema

El sistema debe comportarse como una herramienta de ingeniería de software que prioriza la **eficiencia de recursos** y la **experiencia del usuario (UX)**. Debe ser capaz de autogestionar el flujo de datos entre la nube y el almacenamiento local.

2. Stack Tecnológico Obligatorio

- **Lenguaje:** Dart.
- **Framework:** Flutter (Soporte nativo para Android, iOS y Web).
- **Backend & Auth:** Firebase (Authentication, Cloud Firestore, Cloud Storage, Hosting).
- **Base de Datos Local:** Sistema híbrido utilizando **SQLite (paquete sqflite o drift)** para persistencia offline total de exámenes y resúmenes.
- **IA de Procesamiento:** **Gemini 1.5 Flash API** (priorizada por su amplia capa gratuita y ventana de contexto de 1M de tokens).

3. Lógica de Negocio y Flujo de Usuario

A. Onboarding y Perfil de Aprendizaje

Al iniciar por primera vez, la app debe realizar una encuesta obligatoria:

- **Pregunta:** "¿Cuál es tu método de estudio preferido?"
- **Opciones:** 1. Resúmenes de texto. 2. Cuestionarios y Exámenes de práctica. 3. Flashcards (Tarjetas de memoria). 4. Sistema Híbrido (Todo lo anterior).

B. Módulo de Ingesta (Análisis de Documentos)

- **Formatos soportados:** PDF, Word (.docx), Imágenes (JPG/PNG para OCR) y Texto plano.
- **Procesamiento Asíncrono:** Al subir un archivo, el sistema debe delegar el análisis a la IA en segundo plano (Cloud Functions). La UI debe permanecer fluida y permitir al usuario navegar mientras el proceso ocurre.
- **Notificaciones:** Avisar al usuario cuando el material de estudio esté generado y disponible para uso offline.

C. Sistema de Persistencia (Modo Offline)

1. La IA genera el material de estudio en formato **JSON**.
2. La app descarga el JSON y lo inserta en la base de datos **SQLite** local.
3. **(Nota Crítica)**: Si el usuario pierde la conexión a internet, debe poder acceder a todos los exámenes y resúmenes previamente generados sin ninguna interrupción.

4. Optimización de IA y Gestión de Tokens (MANDATORIO)

(Instrucción de Costo-Eficiencia): Para garantizar que la aplicación sea gratuita de forma ilimitada o de muy bajo costo, el desarrollador debe implementar:

- **Caché de Contexto**: No volver a enviar el mismo documento a la IA. Una vez analizado, el resultado estructurado se almacena localmente.
- **Prompt Engineering**: Solicitar respuestas estrictamente en formato JSON para minimizar el uso de tokens de salida y facilitar el parseo automático.
- **Uso de Gemini 1.5 Flash**: Configurar la API Key para utilizar la capa gratuita de Google AI Studio, manejando los límites de velocidad (Rate Limits) mediante reintentos automáticos con exponencial backoff.

5. Requerimientos de Interfaz (UI/UX)

- **Estilo**: Minimalista, limpio y moderno (Estilo "Productive Academic").
- **Colores**: Paleta suave (pasteles o tonos neutros) para evitar la fatiga visual durante el estudio.
- **Feedback**: Indicadores visuales claros de "Analizando datos...", "Sincronizando..." y "Disponible Offline (Check verde)".

Instrucción Final para la IA: Procede a estructurar el proyecto base en Flutter, configurando los modelos de datos para los usuarios, los documentos subidos y los cuestionarios generados. Genera el código de integración con la API de Gemini siguiendo los parámetros de eficiencia mencionados.